MODULE 01 · DOMAIN 1 - KUBERNETES FUNDAMENTALS

Containers from first principles

What a container really is, and why Kubernetes needs one.

DOMAIN 1 of 4 EXAM WEIGHT 44% MODULE 1 of 35

admin@sthithapragna.com

A container is not a small virtual machine. It is an ordinary process with a restricted view.

The problem containers solve

Meet Quayside, the ferry-booking app we will follow all course.

- **It works on my laptop.** The booking API passes every test, then fails on the server.
- **Dependency drift.** The server has a different Python version and an older system library.
- **Slow, heavy environments.** A new virtual machine per service takes minutes and gigabytes.
- **The fix.** Ship the app together with everything it depends on, as one sealed unit.

> ON THE EXAM
>
> Containers package the app with its dependencies, so it runs the same everywhere.

How we got here

01

Physical servers

One app per box. Wasted capacity, slow to buy.

02

Virtual machines

Many guest operating systems on one host. Better use, still heavy.

03

Containers

Many isolated processes on one kernel. Light and fast.

04

Orchestration

Software that runs thousands of containers across many machines.

ON THE EXAM

Each step fixed the last one's waste. Orchestration fixes the problem containers created: there are now too many to manage by hand.

Containers versus virtual machines

VIRTUAL MACHINE

Its own kernel

- Hypervisor emulates hardware
- Full guest OS in every VM
- Boots in tens of seconds or more
- Strong isolation boundary

CONTAINER

Shares the host kernel

- Kernel features isolate processes
- No guest OS, just app files
- Starts in about a second
- Weaker boundary: one shared kernel

THE TIEBREAKER

The shared kernel is the whole difference. It makes containers fast and dense, and it makes their isolation weaker.

Why the shipping container is the right picture

- **A standard outside.** Every box has the same size and fittings, whatever is inside.
- **The crane does not care.** Ports move boxes without opening them.
- **For software:** the image is the standard box, and any compliant runtime can run it.
- The standard that makes this true is the OCI, covered in module 2.

> ON THE EXAM
>
> Standardise the package, and the machinery that moves it can be generic.

Two kernel features do all the work

- **Namespaces** control what a process can **see**: its own processes, network, hostname, files.
- **Control groups (cgroups)** control what a process can **use**: CPU, memory, process count, I/O.
- A container runtime simply starts a process with both applied, plus an image as its filesystem.

> REMEMBER
>
> Namespaces isolate. Cgroups limit. If an option mixes them up, it is wrong.

The eight Linux namespaces

NAMESPACE	GIVES THE PROCESS ITS OWN...
PID	process IDs, so the app sees itself as PID 1
Network	interfaces, IP addresses, ports and routes
Mount	view of the filesystem and mount points
UTS	hostname
IPC	shared memory and message queues
User	user and group IDs, so root inside need not be root outside
Cgroup	view of its own cgroup tree
Time	boot and monotonic clocks, added in Linux 5.6

ON THE EXAM

In Kubernetes, containers in one Pod share the network namespace. That is why they reach each other on localhost.

One process, two points of view

Illustrative output: the same web server seen from the host and from inside.

```
# On the host
$ ps -eo pid,comm | grep nginx
  48213 nginx

# Inside the container
$ ps -eo pid,comm
      1 nginx
$ hostname
  7f3c9a1e2b4d
```

3 On the host it is just one more process, number 48213.

7 The PID namespace makes it believe it is process 1.

9 The UTS namespace gives it its own hostname: the container ID.

THE TAKEAWAY

Nothing was virtualised. The kernel simply shows the process a different picture.

Control groups: the limits

- **CPU.** A share of processor time, and optionally a hard quota.
- **Memory.** A hard ceiling. Cross it and the kernel kills the process.
- **Process count and I/O.** Stop a fork bomb or a noisy disk user starving neighbours.
- **In Kubernetes**, the requests and limits you set on a container become cgroup settings.

> ON THE EXAM
>
> Kubernetes does not enforce limits itself. The kubelet asks the runtime, which programs the kernel's cgroups.

Hitting a CPU limit versus a memory limit

OVER THE CPU LIMIT

Throttled

- Kernel pauses the process each period
- App gets slower, latency rises
- Container keeps running
- No restart, no error event

OVER THE MEMORY LIMIT

Killed

- Kernel's OOM killer ends the process
- Status shows OOMKilled
- Kubernetes restarts the container
- Repeated kills lead to CrashLoopBackOff

THE TIEBREAKER

CPU is compressible: you can give less and the app just slows. Memory is not: you cannot take back memory in use.

Currency check: cgroup version 2

- **cgroup v2** is the current Linux interface: one unified hierarchy, better memory control.
- **Kubernetes 1.35 (December 2025):** the kubelet refuses to start on a cgroup v1 node by default.
- An operator can still override it with a kubelet setting, but v1 is deprecated and slated for removal.
- Older study material presents v1 and v2 as equal choices. They no longer are.

> ON THE EXAM
>
> If a question asks which cgroup version current Kubernetes expects, the answer is version 2.

An image is **read-only and shared**. A container is that image plus **one thin writable layer**.

An image is a stack of layers

01
Base
Minimal OS files, for example a slim Debian or Alpine.

›

02
Runtime
The language runtime: Python, Java, Node.

›

03
Dependencies
Libraries the app installs.

›

04
App code
Your source or compiled binary.

›

05
Metadata
Which command to run, ports, user, environment.

ON THE EXAM

Layers are cached and shared. Put what changes least at the bottom, and rebuilds only redo the top.

Quayside's booking API, as a Dockerfile

```
1 FROM python:3.12-slim
2 WORKDIR /app
3 COPY requirements.txt .
4 RUN pip install --no-cache-dir -r requirements.txt
5 COPY . .
6 USER 10001
7 EXPOSE 8080
8 CMD ["python", "-m", "booking"]
```

1 The base layer. A slim image keeps size and attack surface down.

3-4 Dependencies first, so the cached layer survives code changes.

5 Code last: it changes most often.

8 The process that becomes PID 1 in the container.

THE TAKEAWAY

A Dockerfile is a text file of build instructions. Order the instructions for the cache.

The writable layer, and why data vanishes

- **Copy-on-write.** A container that changes a file gets a private copy in its writable layer.
- **Cheap to run many.** Ten containers from one image share every read-only layer.
- **Gone on replacement.** Delete or replace the container and its writable layer goes with it.
- **Data that must survive** belongs in a volume, mounted from outside the container.

> ON THE EXAM
>
> Treat containers as disposable. Anything worth keeping lives outside them.

Reading an image reference

PART	EXAMPLE	MEANING
Registry	registry.example.com	Where to pull from. Docker Hub if omitted.
Repository	quayside/booking	The image's name.
Tag	:2.4.1	A movable label. Omitted means "latest".
Digest	@sha256:9f86…	A hash of the content. Can never point elsewhere.

> ON THE EXAM
>
> Tags can move. Digests cannot. Pin by digest when you must know exactly what runs.

When Kubernetes pulls an image

IMAGEPULLPOLICY	BEHAVIOUR
`Always`	Resolve the tag to a digest at every container start; pull if not cached.
`IfNotPresent`	Pull only if the image is not already on the node.
`Never`	Never pull. Start fails if the image is missing.
`Default`	Always for :latest or no tag; IfNotPresent for any other tag or a digest.

ON THE EXAM

While the image downloads, the Pod is Pending. If the pull fails, you will see ErrImagePull, then ImagePullBackOff.

Where Kubernetes fits

- **Kubernetes does not build images.** A CI pipeline does, and pushes them to a registry.
- **Kubernetes does not start processes itself.** It asks a container runtime on each node.
- **What it does:** decide where containers run, keep the right number running, restart failures, connect and update them.
- The unit it schedules is the Pod, a wrapper around one or more containers.

> ON THE EXAM
>
> Orchestration means scheduling, scaling, healing, networking and rolling updates. Building images is not on that list.

> At the evening rush, Quayside's payment worker takes eight seconds per payment instead of one. It never restarts, and no error events appear.
>
> Monitoring shows the container's CPU usage flat at exactly its configured limit. Memory sits at half its limit.

What is the most likely explanation?

A The kernel's out-of-memory killer is ending the process

B The container has exhausted its PID namespace

C The kernel is throttling the container at its CPU limit

D The image layers are being re-downloaded under load

◆ PAUSE AND DECIDE

What is the most likely explanation?

A The kernel's out-of-memory killer is ending the process

Memory is at half its limit, and an OOM kill would restart the container.

B The container has exhausted its PID namespace

A namespace changes what a process sees. It is not a quota that runs out.

C The kernel is throttling the container at its CPU limit

Slow, never restarted, CPU pinned at the limit: the signature of throttling.

D The image layers are being re-downloaded under load

Images are pulled once at start, not while the container runs.

THE SIGNAL

Slow but alive means CPU throttling. Killed and restarted means memory.

> Quayside wants to run forty small services on three Linux servers. Several services need different, conflicting versions of the same library.
>
> Every service runs on Linux. The team wants new copies to start within seconds when traffic rises, with the least overhead per service.

Which approach best meets these requirements?

A Run each service as a container from its own image

B Give each service its own virtual machine

C Install every library version on the shared hosts

D Buy a dedicated physical server per service

◆ PAUSE AND DECIDE

Which approach best meets these requirements?

A Run each service as a container from its own image

Each image carries its own libraries; shared kernel means fast start and low overhead.

B Give each service its own virtual machine

It isolates dependencies, but forty guest OSes are heavy and slow to start.

C Install every library version on the shared hosts

This is the dependency drift problem, made permanent.

D Buy a dedicated physical server per service

Works, but wastes hardware and takes weeks, not seconds.

THE SIGNAL

All Linux, needs density and fast start: containers. Virtual machines are for different kernels or a stronger boundary.

A Quayside developer's container saves uploaded ferry manifests to the /data folder inside the container. Everything works for a week.

After the team deploys a new version of the image, every previously uploaded manifest has disappeared. The new container starts cleanly.

Why did the files disappear?

A The new container's mount namespace hides the old files

B The new image version was pulled using a different tag

C The memory cgroup limit truncated the files on disk

D The files were in the old container's writable layer

◆ PAUSE AND DECIDE

Why did the files disappear?

A. The new container's mount namespace hides the old files

Sounds plausible, but the files are gone, not hidden. The old layer was deleted.

B. The new image version was pulled using a different tag

True but irrelevant. Images never contain runtime uploads.

C. The memory cgroup limit truncated the files on disk

Cgroups limit memory use; they do not edit files on disk.

D. The files were in the old container's writable layer

Deploying replaces the container, and its writable layer goes with it.

THE SIGNAL

Data written inside a container dies with the container. Put it in a volume.

> Two copies of Quayside's web front end use the image quayside/web:latest. One was started last Tuesday; the other started this morning.
>
> They behave differently: only this morning's copy shows the new timetable page. Nobody changed any configuration.

What is the most likely cause?

A Containers on different nodes see different kernel namespaces

B The latest tag was moved to a new image in between

C One copy has modified its read-only image layers

D The two containers were given different CPU limits

◆ PAUSE AND DECIDE

What is the most likely cause?

A. Containers on different nodes see different kernel namespaces

Namespaces isolate each container; they do not change which code runs.

B. The latest tag was moved to a new image in between

Someone pushed a new image under latest between Tuesday and today.

C. One copy has modified its read-only image layers

Read-only layers cannot be changed. Changes go to the writable layer.

D. The two containers were given different CPU limits

Limits affect speed, not which page the app renders.

THE SIGNAL

Two start times plus one movable tag equals two versions. Pin a version tag or a digest.

Module 1 in five lines

- A container is **a process with a restricted view**, sharing the host kernel.
- **Namespaces isolate** what it sees; **cgroups limit** what it uses.
- Over the CPU limit it is **throttled**; over the memory limit it is **OOMKilled**.
- An image is **read-only layers**; the container's writable layer dies with it.
- **Tags move, digests do not.** No tag means latest, and latest means pull Always.

Kubernetes never starts a container itself. So **who does**, and what makes them interchangeable?